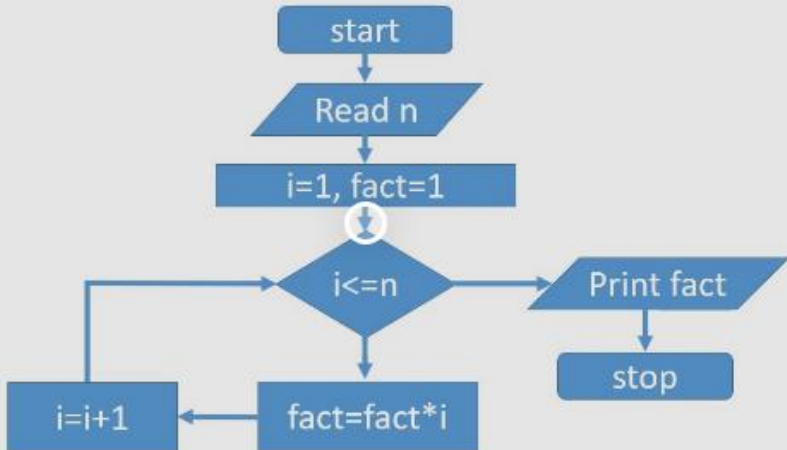
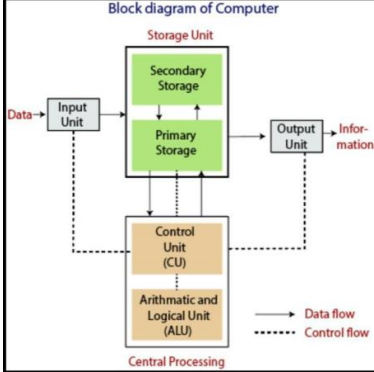


Sahrdaya College of Engineering and Technology, Kodakara		
Answer Key		
APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY SECOND SEMESTER B.TECH DEGREE EXAMINATION, JULY 2021		
Course Code: EST 102		
Course Name: PROGRAMMING IN C		
PART A		
		Answer all questions. Each question carries 3 marks
	Q1	Differentiate between system software and application software.
	Ans	1. System Software is the type of software which is the interface between application software and system. On other hand Application Software is the type of software which runs as per user request. It runs on the platform which is provide by system software. 2. System software is used for operating computer hardware. On other hand Application software is used by user to perform specific task. 3. Some examples of system software's are compiler, assembler, debugger, driver, etc. Some examples of application software's are word processor, web browser, media player, etc.
	Q2	Write an algorithm to find the largest of three numbers
	Ans	Step 1: start. Step 2: input a, b, c Step 3: let Max=a. Step 4: is b > Max? then Max = b Step 5: is c > max? then Max = c Step6: print Max Step7: stop
	Q3	What is the difference between assignment and equality operators?
	Ans	The "==" is an assignment operator is used to assign the value on the right to the variable on the left. The '==' operator checks whether the two given operands are equal or not.
	Q4	What is a static variable? When should it be used?
	Ans	Static variables are initialized only once. The compiler persists with the variable till the end of the program. Static variables can be defined inside or outside the function. They are local to the block. The default value of static variables is zero. The static variables are alive till the execution of the program. 1. Static defined local variables do not lose their value between function calls. In other words they are global variables, but scoped to the local function they are defined in. 2. Static global variables are not visible outside of the C file they are defined in.
	Q5	Write a C program to find length of a string without using string handling functions.
	Ans	<pre>#include <stdio.h> void main() { char str[100],i; printf("Enter a string: \n");</pre>

		<pre>scanf("%s",str); for(i=0; str[i]!='\0'; ++i); printf("\nLength of input string: %d",i); }</pre>
	Q6	What is an array? Illustrate using an example, how a single dimensional array is initialised.
	Ans	An array is a collection of similar data elements stored at contiguous memory locations. It is the simplest data structure where each data element can be accessed directly by only using its index number. <pre>int marks[5];</pre> array marks[5] defines the marks scored by a student in 5 different subjects where each subject marks are located at a particular location in the array i.e. marks[0] denotes the marks scored in first subject, marks[1] denotes the marks scored in 2nd subject and so on.
	Q7	Differentiate between structure and union using an example.
	Ans	A structure is a user-defined data type available in C that allows to combining data items of different kinds. Structures are used to represent a record. Each member is assigned unique storage. To define a structure, you must use the keyword struct . A union is a special data type available in C that allows storing different data types in the same memory location. Memory allotted is shared by union members. You can define a union with many members, but only one member can contain a value at any given time. Unions provide an efficient way of using the same memory location for multiple purposes. To define a union, you must use the keyword union.
	Q8	Illustrate the purpose of return statement using an example.
	Ans	A return statement ends the execution of a function, and returns control to the calling function. Execution resumes in the calling function at the point immediately following the call. A return statement can return a value to the calling function. <pre>#include <stdio.h> void Print() { printf("Welcome to Programming"); } int main() { Print(); return 0; }</pre>
	Q9	Differentiate between char name[] and char *name in C.
	Ans	One is an array that refers to an array of characters. The other is a pointer to a character.
	Q10	Explain the use of fseek() function.
	Ans	<pre>int fseek(FILE *pointer, long int offset, int position)</pre> pointer: pointer to a FILE object that identifies the stream. offset: number of bytes to offset from position position: position from where offset is added. The fseek() function seeks in an open file. This function moves the file pointer from its current position to a new position, forward or backward, specified by the number of bytes.
PART B		
<i>Answer any one full question from each module. Each question carries 14 marks</i>		
	Q11a	Draw a flowchart to find the factorial of a number.

Ans	 <pre> graph TD Start([start]) --> Read[/Read n/] Read --> Init[i=1, fact=1] Init --> Cond{i <= n} Cond -- Yes --> FactCalc[fact=fact*i] FactCalc --> IncI[i=i+1] IncI --> Cond Cond -- No --> Print[/Print fact/] Print --> Stop([stop]) </pre>	
Q11b	With the help of a neat diagram explain the functional units of a computer.	
Ans	Block diagram of Computer  1. Input: This is the process of entering data and programs in to the computer system. You should know that computer is an electronic machine like any other machine which takes as inputs raw data and performs some processing giving out processed data. Therefore, the input unit takes data from us to the computer in an organized manner for processing. 2. Storage: The process of saving data and instructions permanently is known as storage. Data has to be fed into the system before the actual processing starts. It is because the processing speed of Central Processing Unit (CPU) is so fast that the data has to be provided to CPU with the same speed. Therefore the data is first stored in the storage unit for faster access and processing. This storage unit or the primary storage of the computer system is designed to do the above functionality. It provides space for storing data and instructions. The storage unit performs the following major functions: - All data and instructions are stored here before and after processing. - Intermediate results of processing are also stored here. 3. Processing: The task of performing operations like arithmetic and logical operations is called processing. The Central Processing Unit (CPU) takes data and instructions from the storage unit and makes all sorts of calculations based on the instructions given and the type of data provided. It is then sent back to the storage unit.	

	4. Output: This is the process of producing results from the data for getting useful information. Similarly the output produced by the computer after processing must also be kept somewhere inside the computer before being given to you in human readable form. Again the output is also stored inside the computer for further processing. 5. Control: The manner how instructions are executed and the above operations are performed. Controlling of all operations like input, processing and output are performed by control unit. It takes care of step by step processing of all operations inside the computer.																																				
Q12a	List five important registers in CPU. Also state the purpose of each register.																																				
Ans	<table><thead><tr><th>Register</th><th>Symbol</th><th>Number of bits</th><th>Function</th></tr></thead><tbody><tr><td>Data register</td><td>DR</td><td>16</td><td>Holds memory operand</td></tr><tr><td>Address register</td><td>AR</td><td>12</td><td>Holds address for the memory</td></tr><tr><td>Accumulator</td><td>AC</td><td>16</td><td>Processor register</td></tr><tr><td>Instruction register</td><td>IR</td><td>16</td><td>Holds instruction code</td></tr><tr><td>Program counter</td><td>PC</td><td>12</td><td>Holds address of the instruction</td></tr><tr><td>Temporary register</td><td>TR</td><td>16</td><td>Holds temporary data</td></tr><tr><td>Input register</td><td>INPR</td><td>8</td><td>Carries input character</td></tr><tr><td>Output register</td><td>OUTR</td><td>8</td><td>Carries output character</td></tr></tbody></table>	Register	Symbol	Number of bits	Function	Data register	DR	16	Holds memory operand	Address register	AR	12	Holds address for the memory	Accumulator	AC	16	Processor register	Instruction register	IR	16	Holds instruction code	Program counter	PC	12	Holds address of the instruction	Temporary register	TR	16	Holds temporary data	Input register	INPR	8	Carries input character	Output register	OUTR	8	Carries output character
Register	Symbol	Number of bits	Function																																		
Data register	DR	16	Holds memory operand																																		
Address register	AR	12	Holds address for the memory																																		
Accumulator	AC	16	Processor register																																		
Instruction register	IR	16	Holds instruction code																																		
Program counter	PC	12	Holds address of the instruction																																		
Temporary register	TR	16	Holds temporary data																																		
Input register	INPR	8	Carries input character																																		
Output register	OUTR	8	Carries output character																																		
Q12b	Write algorithm and draw flowchart to perform swapping of two numbers.																																				
Ans	- Let n1 and n2 be the two numbers. - int temp; - temp = n1 - n1 = n2 - n2 = temp - Display n1 and n2 <pre>graph TD; START([START]) --> Init[N1 ← 0, N2 ← 0, TEMP ← 0]; Init --> Read[/Read N1, N2/]; Read --> Swap[TEMP ← N1, N1 ← N2, N2 ← TEMP]; Swap --> Print[/Print N1, N2/]; Print --> STOP([STOP]);</pre>																																				
Q13a	Explain arithmetic, logical and bitwise operators with examples.																																				

Ans	<table border="1"> <thead> <tr> <th>Operator</th><th>Type</th></tr> </thead> <tbody> <tr> <td>Unary operator + , - , ~</td><td>Unary operator</td></tr> <tr> <td>Binary operator + , - , * , / , %</td><td>Arithmetic operator</td></tr> <tr> <td>< , <= , > , >= , == , !=</td><td>Relational operator</td></tr> <tr> <td>&& , , !</td><td>Logical operator</td></tr> <tr> <td>& , , << , >> , ~ , ^</td><td>Bitwise operator</td></tr> <tr> <td>= , += , -= , *= , /= , %=</td><td>Assignment operator</td></tr> <tr> <td>?:</td><td>Ternary or conditional operator</td></tr> </tbody> </table>	Operator	Type	Unary operator + , - , ~	Unary operator	Binary operator + , - , * , / , %	Arithmetic operator	< , <= , > , >= , == , !=	Relational operator	&& , , !	Logical operator	& , , << , >> , ~ , ^	Bitwise operator	= , += , -= , *= , /= , %=	Assignment operator	?:	Ternary or conditional operator
Operator	Type																
Unary operator + , - , ~	Unary operator																
Binary operator + , - , * , / , %	Arithmetic operator																
< , <= , > , >= , == , !=	Relational operator																
&& , , !	Logical operator																
& , , << , >> , ~ , ^	Bitwise operator																
= , += , -= , *= , /= , %=	Assignment operator																
?:	Ternary or conditional operator																
Q13b	Write a C Program to check if a given number is a strong number or not. A strong number is a number in which the sum of the factorial of the digits is equal to the number itself.																
Ans	<pre> #include <stdio.h> int main() { int i, originalNum, num, lastDigit, sum; long fact; printf("Enter any number to check Strong number: "); scanf("%d", &num); originalNum = num; sum = 0; /* Find sum of factorial of digits */ while(num > 0) { /* Get last digit of num */ lastDigit = num % 10; /* Find factorial of last digit */ fact = 1; for(i=1; i<=lastDigit; i++) { fact = fact * i; } /* Add factorial to sum */ sum = sum + fact; num = num / 10; } /* Check Strong number condition */ if(sum == originalNum) { printf("%d is STRONG NUMBER", originalNum); } else { printf("%d is NOT STRONG NUMBER", originalNum); } return 0; } </pre>																
Q14a	Write C program to convert the given decimal number into binary number.																

Ans	<pre> #include <stdio.h> #include <math.h> long decimalToBinary(int decimalnum) { long binarynum = 0; int rem, temp = 1; while (decimalnum!=0) { rem = decimalnum%2; decimalnum = decimalnum / 2; binarynum = binarynum + rem*temp; temp = temp * 10; } return binarynum; } int main() { int decimalnum; printf("Enter a Decimal Number: "); scanf("%d", &decimalnum); printf("Equivalent Binary Number is: %ld", decimalToBinary(decimalnum)); return 0; } </pre>
Q14b	What do you mean by Formatted Input? Explain in detail the prototype of 'scanf()' function in C including its argument list and return type.
Ans	Several library functions help you convert data values from text sequences that are generally readable by people to encoded internal representations. You provide a format string as the value of the format argument to each of these functions, hence the term formatted input. The functions fall into two categories: The byte scan functions (declared in <stdio.h>) convert sequences of type char to internal representations, and help you scan such sequences that you read: fscanf, scanf, sscanf, vfscanf, vscanf, and vsscanf. For these function, a format string is a multibyte string that begins and ends in the initial shift state. The function scanf() is used for formatted input from standard input and provides many of the conversion facilities of the function printf(). Syntax <pre>scanf (format, num1, num2,.....);</pre> The function scanf() reads and converts characters from the standards input depending on the format specification string and stores the input in memory locations represented by the other arguments (num1, num2,...). For Example: <pre>scanf(" %c %d",&Name, &Roll No);</pre>
Q15a	Explain any 4 string handling functions in C programming.

Ans	Function	Syntax (or) Example	Description
	strcpy()	strcpy(string1, string2)	Copies string2 value into string1
	strncpy()	strncpy(string1, string2, 5)	Copies first 5 characters string2 into string1
	strlen()	strlen(string1)	returns total number of characters in string1
	strcat()	strcat(string1,string2)	Appends string2 to string1
	strncat()	strncpy(string1, string2, 4)	Appends first 4 characters of string2 to string1
	strcmp()	strcmp(string1, string2)	Returns 0 if string1 and string2 are the same; less than 0 if string1<string2; greater than 0 if string1>string2
	strncmp()	strncmp(string1, string2, 4)	Compares first 4 characters of both string1 and string2
	strcmpli()	strcmpli(string1,string2)	Compares two strings, string1 and string2 by ignoring case (upper or lower)
	stricmp()	stricmp(string1, string2)	Compares two strings, string1 and string2 by ignoring case (similar to strcmpli())
Q15b	Write a C program to perform linear search on an array of numbers		
Ans	<pre>#include <stdio.h> int main() { int array[100], search, c, n; printf("Enter number of elements in array\n"); scanf("%d", &n); printf("Enter %d integer(s)\n", n); for (c = 0; c < n; c++) scanf("%d", &array[c]); printf("Enter a number to search\n"); scanf("%d", &search); for (c = 0; c < n; c++) { if (array[c] == search) /* If required element is found */ { printf("%d is present at location %d.\n", search, c+1); break; } } if (c == n) printf("%d isn't present in the array.\n", search); return 0; }</pre>		
Q16a	Write a C program to find second largest element in an array.		
Ans	<pre>#include <stdio.h> void main(){ int arr1[50],n,i,j=0,lrg,lrg2nd; printf("\n\nFind the second largest element in an array :\n"); printf("-----\n"); printf("Input the size of array : "); scanf("%d", &n); /* Stored values into the array*/ printf("Input %d elements in the array :\n",n); for(i=0;i<n;i++)</pre>		

	<pre> { printf("element - %d : ",i); scanf("%d",&arr1[i]); } /* find location of the largest element in the array */ lrg=0; for(i=0;i<n;i++) { if(lrg<arr1[i]) { lrg=arr1[i]; j = i; } } /* ignore the largest element and find the 2nd largest element in the array */ lrg2nd=0; for(i=0;i<n;i++) { if(i==j) { i++; /* ignoring the largest element */ i--; } else { if(lrg2nd<arr1[i]) { lrg2nd=arr1[i]; } } } printf("The Second largest element in the array is : %d \n\n", lrg2nd); } </pre>
Q16b	Write a C program to check whether a string is palindrome or not without using string handling functions.
Ans	<pre> #include <stdio.h> #include <string.h> void main() { char string[25], reverse_string[25] = {'\0'}; int i, length = 0, flag = 0; printf("Enter a string \n"); gets(string); /* keep going through each character of the string till its end */ for (i = 0; string[i] != '\0'; i++) { length++; } printf("The length of the string '%s' = %d\n", string, length); for (i = length - 1; i >= 0 ; i--) { </pre>

	<pre>reverse_string[length - i - 1] = string[i]; } /* Check if the string is a Palindrome */ for (flag = 1, i = 0; i < length ; i++) { if (reverse_string[i] != string[i]) flag = 0; } if (flag == 1) printf ("%s is a palindrome \n", string); else printf ("%s is not a palindrome \n", string); }</pre>																								
Q17a	What are different storage classes in C? Give examples for each.																								
Ans	<table><tr><th>Storage Class</th><th>Declaration Location</th><th>Scope (Visibility)</th><th>Lifetime (Alive)</th></tr><tr><td>auto</td><td>Inside a function/block</td><td>Within the function/block</td><td>Until the function/block completes</td></tr><tr><td>register</td><td>Inside a function/block</td><td>Within the function/block</td><td>Until the function/block completes</td></tr><tr><td>extern</td><td>Outside all functions</td><td>Entire file plus other files where the variable is declared as extern</td><td>Until the program terminates</td></tr><tr><td>static (local)</td><td>Inside a function/block</td><td>Within the function/block</td><td>Until the program terminates</td></tr><tr><td>static (global)</td><td>Outside all functions</td><td>Entire file in which it is declared</td><td>Until the program terminates</td></tr></table> <pre>int count ; extern void write_extern(); void main() { count = 5; write_extern(); } extern int count; void write_extern(void) { printf("count is %d\n", count); } void main() { static int i; printf("Value=%d",i); }</pre>	Storage Class	Declaration Location	Scope (Visibility)	Lifetime (Alive)	auto	Inside a function/block	Within the function/block	Until the function/block completes	register	Inside a function/block	Within the function/block	Until the function/block completes	extern	Outside all functions	Entire file plus other files where the variable is declared as extern	Until the program terminates	static (local)	Inside a function/block	Within the function/block	Until the program terminates	static (global)	Outside all functions	Entire file in which it is declared	Until the program terminates
Storage Class	Declaration Location	Scope (Visibility)	Lifetime (Alive)																						
auto	Inside a function/block	Within the function/block	Until the function/block completes																						
register	Inside a function/block	Within the function/block	Until the function/block completes																						
extern	Outside all functions	Entire file plus other files where the variable is declared as extern	Until the program terminates																						
static (local)	Inside a function/block	Within the function/block	Until the program terminates																						
static (global)	Outside all functions	Entire file in which it is declared	Until the program terminates																						

		<pre> void main() { register int i; printf("Value=%d",i); } void main() { auto int i; printf("Value=%d",i); } </pre>
	Q17b	Write a C program to: (i) Create a structure with fields: Name, Address, Date of birth. (ii) Read the above details for five students from user and display the details
	Ans	<pre> #include <stdio.h> struct student { char Name[50]; int roll; float marks; } s[5]; int main() { int i; printf("Enter information of students:\n"); // storing information for (i = 0; i < 5; ++i) { s[i].roll = i + 1; printf("\nFor roll number%d,\n", s[i].roll); printf("Enter name: "); scanf("%s", s[i].Name); printf("Enter marks: "); scanf("%f", &s[i].marks); } printf("Displaying Information:\n\n"); // displaying information for (i = 0; i < 5; ++i) { printf("\nRoll number: %d\n", i + 1); printf("Name: "); puts(s[i].Name); printf("Marks: %.1f", s[i].marks); printf("\n"); } return 0; } </pre>
	Q18a	What is recursion? Write a C program to display Fibonacci series using recursive function.

Ans	Recursion is the process which comes into existence when a function calls a copy of itself to work on a smaller problem. Any function which calls itself is called recursive function, and such function calls are called recursive calls. <pre> #include<stdio.h> int fibonacci(int); void main () { int n,f; printf("Enter the value of n?"); scanf("%d",&n); f = fibonacci(n); printf("%d",f); } int fibonacci (int a) { if (a==0) { return 0; } else if (a == 1) { return 1; } else { return fibonacci(a-1)+fibonacci(a-2); } } </pre>
Q18b	Write a C program to sort N numbers using functions.
Ans	<pre> #include<stdio.h> /* Function prototype */ void asc_sort(int a[100], int n); void main() { int a[100], i, n; printf("Enter n:\n"); scanf("%d", &n); /* Reading array */ for(i=0;i<n;i++) { printf("a[%d]=",i); scanf("%d", &a[i]); } /* Function Call */ asc_sort(a,n); </pre>

	<pre> /* Displaying sorted array */ printf("Array in ascending order is:\n"); for(i=0;i< n;i++) { printf("%d\t", a[i]); } } /* Function definition for asc_sort */ void asc_sort(int a[10], int n) { int i, j, temp; for(i=0;i< n-1;i++) { for(j=i+1;j< n;j++) { if(a[i]>a[j]) { temp = a[i]; a[i] = a[j]; a[j] = temp; } } } } </pre>										
Q19a	Explain any 5 file handling functions in C.										
Ans	<table border="1"> <thead> <tr> <th>File operation</th><th>Declaration & Description</th></tr> </thead> <tbody> <tr> <td>fopen() - To open a file</td><td> Declaration: FILE *fopen (const char *filename, const char *mode) fopen() function is used to open a file to perform operations such as reading, writing etc. In a C program, we declare a file pointer and use fopen() as below. fopen() function creates a new file if the mentioned file name does not exist. <pre>FILE *fp; fp=fopen ("filename", "mode");</pre> Where, fp - file pointer to the data type "FILE". filename - the actual file name with full path of the file. mode - refers to the operation that will be performed on the file. Example: r, w, a, r+, w+ and a+. Please refer below the description for these mode of operations. </td></tr> <tr> <td>fclose() - To close a file</td><td> Declaration: int fclose(FILE *fp); fclose() function closes the file that is being pointed by file pointer fp. In a C program, we close a file as below. <pre>fclose (fp);</pre> </td></tr> <tr> <td>fgets() - To read a file</td><td> Declaration: char *fgets(char *string, int n, FILE *fp) fgets function is used to read a file line by line. In a C program, we use fgets function as below. <pre>fgets (buffer, size, fp);</pre> where, buffer - buffer to put the data in. size - size of the buffer fp - file pointer </td></tr> <tr> <td>fprintf() - To write into a file</td><td> Declaration: <pre>int fprintf(FILE *fp, const char *format, ...);</pre> fprintf() function writes string into a file pointed by fp. In a C program, we write string into a file as below. fprintf (fp, "some data"); or fprintf (fp, "text %d", variable_name); </td></tr> </tbody> </table>	File operation	Declaration & Description	fopen() - To open a file	Declaration: FILE *fopen (const char *filename, const char *mode) fopen() function is used to open a file to perform operations such as reading, writing etc. In a C program, we declare a file pointer and use fopen() as below. fopen() function creates a new file if the mentioned file name does not exist. <pre>FILE *fp; fp=fopen ("filename", "mode");</pre> Where, fp - file pointer to the data type "FILE". filename - the actual file name with full path of the file. mode - refers to the operation that will be performed on the file. Example: r, w, a, r+, w+ and a+. Please refer below the description for these mode of operations.	fclose() - To close a file	Declaration: int fclose(FILE *fp); fclose() function closes the file that is being pointed by file pointer fp. In a C program, we close a file as below. <pre>fclose (fp);</pre>	fgets() - To read a file	Declaration: char *fgets(char *string, int n, FILE *fp) fgets function is used to read a file line by line. In a C program, we use fgets function as below. <pre>fgets (buffer, size, fp);</pre> where, buffer - buffer to put the data in. size - size of the buffer fp - file pointer	fprintf() - To write into a file	Declaration: <pre>int fprintf(FILE *fp, const char *format, ...);</pre> fprintf() function writes string into a file pointed by fp. In a C program, we write string into a file as below. fprintf (fp, "some data"); or fprintf (fp, "text %d", variable_name);
File operation	Declaration & Description										
fopen() - To open a file	Declaration: FILE *fopen (const char *filename, const char *mode) fopen() function is used to open a file to perform operations such as reading, writing etc. In a C program, we declare a file pointer and use fopen() as below. fopen() function creates a new file if the mentioned file name does not exist. <pre>FILE *fp; fp=fopen ("filename", "mode");</pre> Where, fp - file pointer to the data type "FILE". filename - the actual file name with full path of the file. mode - refers to the operation that will be performed on the file. Example: r, w, a, r+, w+ and a+. Please refer below the description for these mode of operations.										
fclose() - To close a file	Declaration: int fclose(FILE *fp); fclose() function closes the file that is being pointed by file pointer fp. In a C program, we close a file as below. <pre>fclose (fp);</pre>										
fgets() - To read a file	Declaration: char *fgets(char *string, int n, FILE *fp) fgets function is used to read a file line by line. In a C program, we use fgets function as below. <pre>fgets (buffer, size, fp);</pre> where, buffer - buffer to put the data in. size - size of the buffer fp - file pointer										
fprintf() - To write into a file	Declaration: <pre>int fprintf(FILE *fp, const char *format, ...);</pre> fprintf() function writes string into a file pointed by fp. In a C program, we write string into a file as below. fprintf (fp, "some data"); or fprintf (fp, "text %d", variable_name);										

	fscanf() function – To read a file Syntax: <pre> int fscanf(FILE *stream, const char *format [, argument, ...]) #include <stdio.h> main(){ FILE *fp; char buff[255]; //creating char array to store data of file fp = fopen("file.txt", "r"); while(fscanf(fp, "%s", buff)!=EOF){ printf("%s ", buff); } fclose(fp); } </pre>
Q19b	Write a C program to reverse a string using pointers.
Ans	<pre> #include <stdio.h> int main() { char str1[50]; char revstr[50]; char *stptr = str1; char *rvptr = revstr; int i=-1; printf("\n\n Pointer : Print a string in reverse order :\n"); printf("-----\n"); printf(" Input a string : "); scanf("%s",str1); while(*stptr) { stptr++; i++; } while(i>=0) { stptr--; *rvptr = *stptr; rvptr++; --i; } *rvptr='\0'; printf(" Reverse of the string is : %s\n\n",revstr); return 0; } </pre>
Q20a	Differentiate between array of pointers and pointer to an array.

Ans	Parameters	Pointer to an Array	Array of Pointers
	Uses and Purposes	A user creates a pointer for storing the address of any given array.	A user creates an array of pointers that basically acts as an array of multiple pointer variables.
	Alternative Names	It is alternatively known as an array pointer.	These are alternatively known as pointer arrays.
	Allocation	One can allocate these during the run time.	One can allocate these during the compile time.
	Initialization at Definition	You cannot initialize a pointer to the definition.	You can easily initialize an array at the definition level.
	Nature	It is dynamic in nature.	It is static in nature.
	Resizing	One can easily resize the allocated memory of a pointer later at any given time.	Once we declare the size of an array, we cannot resize it any time we want according to our requirements.
	Type of Storage	A typical pointer variable is capable of storing only a single variable within.	The size of any given array decides the total number of variables that it can store within.
Q20b	Write a C program to count number of lines in a text file.		
Ans	<pre> #include<iostream> using namespace std; #define FILENAME "test.txt" int main() { FILE *fp; char ch; int linesCount=0; //open file in read mode fp=fopen(FILENAME,"r"); if(fp==NULL) { printf("File \"%s\" does not exist!!\n",FILENAME); return -1; } //read character by character and check for new line while((ch=fgetc(fp))!=EOF) { if(ch=='\n') linesCount++; } //close the file fclose(fp); //print number of lines printf("Total number of lines are: %d\n",linesCount); return 0; } </pre>		